Introduction to Machine Learning M146 UCLA Spring Quarter 2026
Prof. Vwani Roychowdhury 8th May 2026

TAs: Kartik Sharma, M. Yigit Turali

Homework #3

Due: 22nd May 2026, Friday, before 11:59 pm

Problem 1 (The K-Means Objective and Lloyd's Algorithm)

The standard k-means objective is the within-cluster sum of squared distances:

$$J(\boldsymbol{\mu}, z) = \sum_{n=1}^{N} \|x_n - \mu_{z_n}\|^2,$$

where each $x_n \in \mathbb{R}^d$, each $z_n \in \{1, \ldots, k\}$ is a cluster assignment, and each $\mu_c \in \mathbb{R}^d$ is a cluster centroid. Lloyd's algorithm alternates two steps:

- **Assignment step:** hold the centroids fixed; update each z_n to minimise J.
- **Update step:** hold the assignments fixed; update each centroid μ_c to minimise J.

(a) **Optimal centroid given assignments.**

Fix the assignments $\{z_n\}$. Show that the value of μ_c that minimises J is the mean of the points currently assigned to cluster c:

$$\mu_c^* = \frac{1}{|C_c|} \sum_{n\,:\,z_n = c} x_n, \qquad C_c = \{n : z_n = c\}.$$

Hint: take $\nabla_{\mu_c} J = 0$ and solve.

(b) **Optimal assignment given centroids.**

Fix the centroids $\{\mu_c\}$. Show that the value of z_n that minimises J is

$$z_n^* = \arg \min_{c \in \{1,\ldots,k\}} \|x_n - \mu_c\|^2.$$

Hint: the per-point contribution $\|x_n - \mu_{z_n}\|^2$ is the only term that depends on z_n, and the sum decouples across n.

(c) **Monotone decrease.**

Using parts (1) and (2), argue that Lloyd's algorithm produces a sequence of iterates $J^{(0)} \geq J^{(1)} \geq J^{(2)} \geq \ldots$ that is non-increasing, and that the algorithm converges in finitely many steps.

Hint: there are only finitely many possible assignments $\{z_n\}$, so a strictly decreasing sequence cannot continue forever; the algorithm halts as soon as no assignment changes.

(d) **Local vs. global optimum.**

In one or two sentences, explain why the J that Lloyd's algorithm converges to is only guaranteed to be a *local* minimum, and why this motivates running the algorithm multiple times with different random initialisations. You will see this empirically in Problem 4(g) of this homework.

Problem 2 (Principal Component Analysis: Variance Maximisation and SVD)

Let $X \in \mathbb{R}^{N \times d}$ be a data matrix and let X_c denote its centered version (each column has mean zero). The sample covariance matrix is

$$C = \frac{1}{N-1} X_c^T X_c \in \mathbb{R}^{d \times d}.$$

(a) **The first principal component maximises projected variance.**

Consider a unit vector $v \in \mathbb{R}^d$ and the projected data $z_n = v^T(x_n - \bar{x})$.

i. Show that the sample variance of $z_1, \ldots, z_N$ equals $v^T C v$.

ii. Form the Lagrangian

$$L(v, \lambda) = v^T C v - \lambda(v^T v - 1)$$

and set $\nabla_v L = 0$. Conclude that the optimal v satisfies $Cv = \lambda v$—i.e. v is an eigenvector of C.

iii. Among all eigenvectors of C, which one maximises $v^T C v$?

(b) **SVD and eigendecomposition give the same principal axes.**

Suppose the SVD of the centered data is $X_c = U \Sigma V^T$ with $U \in \mathbb{R}^{N \times r}$, $\Sigma \in \mathbb{R}^{r \times r}$ diagonal, and $V \in \mathbb{R}^{d \times r}$, where $r = \min(N, d)$.

i. Show that $X_c^T X_c = V \Sigma^2 V^T$.

ii. Conclude that the columns of V are eigenvectors of $C = X_c^T X_c/(N-1)$, and that the i-th eigenvalue of C is $\lambda_i = \sigma_i^2/(N-1)$, where σ_i is the i-th singular value.

iii. Briefly, why is the SVD route preferred numerically when d is very large (e.g. $d =$ 20,531 as in Problem 5 of this homework)?

You will verify this equivalence empirically in Problem 5(c) of this homework.

Problem 3 (Penguin Species Discovery with K-Means)

Imagine you are a research assistant on a long-running ecology study at Palmer Station, Antarctica. The team has spent years measuring penguins on three nearby islands—Biscoe, Dream, and Torgersen—collecting bill, flipper, and body-mass measurements on every bird they can catch. The senior researcher walks into the lab one morning and says: "forget what we know about which species is which. Just from the morphology, can we tell that there are distinct *groups* of penguins out there? And if there are, how many?" No labels. Just measurements. This is exactly what **unsupervised clustering** is for.

In this problem you will implement **k-means from scratch** in NumPy and use it to discover penguin groups. Along the way you will see why initialization matters, how to choose k when no one tells you, and how to evaluate clustering quality without ground-truth labels. At the end, the lab notebook does have species labels recorded—so we can check, after the fact, how well our unsupervised clustering recovered them.

Tools. `pandas` for loading and exploring the data, `matplotlib`/`seaborn` for plotting. **All clustering code must use NumPy**—no `sklearn.cluster`. You may use `sklearn.metrics.silhouette_score` for the silhouette evaluation in part (f).

Dataset. Download the *Palmer Penguins* dataset:

https://github.com/allisonhorst/palmerpenguins/

The file `penguins.csv` contains 344 rows—one per penguin. The features are:

Feature	Name	Description
	`species`	Adelie, Chinstrap, or Gentoo (held aside as ground truth)
	`island`	Biscoe, Dream, or Torgersen
x_1	`bill_length_mm`	Bill length, mm
x_2	`bill_depth_mm`	Bill depth, mm
x_3	`flipper_length_mm`	Flipper length, mm
x_4	`body_mass_g`	Body mass, g
	`sex`	male / female (some missing)
	`year`	Observation year

There is no target column, but `species` is a known label that we will *not* use during clustering and treat later as ground truth ("did our clustering rediscover the three species?").

(a) **Exploratory Data Analysis.**

(i) **Basics. [3 pts]** Load `penguins.csv` and print its shape and `.describe()`. How many penguins are in each `species`? On each `island`?

(ii) **Missing values. [3 pts]** Report the number of missing values per column. Drop rows with any `NaN` in the four numerical morphological features. How many penguins remain?

(iii) **Are there visible clusters? [5 pts]** Make a scatter plot of `flipper_length_mm` (x-axis) vs `bill_length_mm` (y-axis). Colour points by `species`. Do you see three blobs by eye? Which two species are most similar in this 2D projection?
Why: clustering algorithms can only find structure that is in the data. Looking at the scatter plot first builds expectations: any reasonable $k = 3$ clustering should approximately recover what your eye sees.

(b) **Preprocessing.**

(i) **Pick the features. [2 pts]** For the rest of this problem, work with the four numerical morphological features

`bill_length_mm`, `bill_depth_mm`, `flipper_length_mm`, `body_mass_g`.

Drop the categorical columns and the `species` column (which we will hold aside as ground truth).

(ii) **Why standardize. [2 pts]** `body_mass_g` is in the thousands; `bill_depth_mm` is in single-digit millimetres. k-means uses Euclidean distance, so without scaling, body mass would completely dominate the distance computation and the algorithm would basically ignore the bill and flipper measurements. Briefly explain this in 1–2 sentences.

(iii) **Standardize. [3 pts]** Standardize each of the four features to zero mean and unit variance using its training-data statistics. Print the resulting means and standard deviations.

(c) **Implement k-means from scratch.**

Recall (and verify against your work in Problem 1 of this homework): k-means iteratively minimises the within-cluster sum of squared distances

$$J(\boldsymbol{\mu}, z) = \sum_{n=1}^{N} \|x_n - \mu_{z_n}\|^2.$$

Lloyd's algorithm alternates the two steps you derived in Problem 1(1)–(2):

- **Assignment step (Problem 1(2)):** $z_n = \arg\min_c \|x_n - \mu_c\|^2$.
- **Update step (Problem 1(1)):** $\mu_c = \frac{1}{|C_c|} \sum_{n:z_n=c} x_n$.

Repeat until assignments stop changing (or until a maximum number of iterations).

(i) **Implement. [8 pts]** Write a function with signature

```
def kmeans(X, k, max_iter=100, seed=0):
    # returns (centroids, labels, inertia)
```

Use random initialization: pick k random rows of `X` (without replacement) as the initial centroids, with the RNG seeded by `seed`. Track J each iteration. Stop when assignments stop changing or when `max_iter` is reached. **Return** the final centroids, the final cluster labels, and the final inertia J.

(ii) **Sanity check on the objective. [3 pts]** Lloyd's algorithm should never *increase* J from one iteration to the next. Plot J as a function of iteration on a single example run with $k = 3$ and verify the monotone-decrease property.

(d) **Cluster and validate.**

(i) **[3 pts]** Run your k-means with $k = 3$ on the four standardized features. Report the final inertia.

(ii) **Did we rediscover the species? [5 pts]** Cross-tabulate your $k = 3$ cluster labels against the true `species` column (use `pandas.crosstab`). Compute the *clustering purity*: assign each cluster to its majority-class species, then count how many penguins ended up in their majority cluster. Report the purity as a fraction.
Why this matters: we did not give k-means the species labels, but a high purity tells us the morphological measurements alone carry essentially the same information as the pathologist-assigned species. That is exactly the question the senior researcher was asking.

(iii) **Cluster profiles in original units. [5 pts]** Inverse-transform the centroids back to the original measurement scale by undoing the standardization (multiply by σ, add μ). Display the centroids in a small table indexed by cluster. In one sentence per cluster, describe "what kind of penguin" each centroid represents in plain English (e.g. "large body, deep bill, long flipper").

(iv) **Visualize. [3 pts]** Reproduce the (`flipper_length_mm`, `bill_length_mm`) scatter plot from part (a)(iii), but colour points by *cluster label* this time. Does it look like the species-coloured version? Where does it differ?

(e) **The elbow method—how many clusters?**

The senior researcher's question was "how many groups are there?"—not necessarily three. The simplest heuristic is the **elbow method**: run k-means for a range of k values, plot

the final inertia $J(k)$, and look for the "elbow"—the value of k after which adding more clusters stops giving a meaningful drop in J.

(i) **[5 pts]** Run your k-means for $k \in \{1, 2, \ldots, 10\}$. For each k, average the final inertia over 10 random initializations (different seeds). Plot mean inertia vs k.

(ii) **[3 pts]** Identify the elbow. Where does the curve clearly bend? Is this consistent with the number of species?
Why average over seeds: a single bad initialization can leave k-means stuck in a poor local minimum, which would make the inertia curve jagged.

(f) **Silhouette score—a more honest evaluation.**

Recall: The **silhouette coefficient** of a single point x_n is

$$s_n = \frac{b_n - a_n}{\max(a_n, b_n)},$$

where a_n is the mean distance from x_n to other points in its own cluster, and b_n is the mean distance from x_n to points in the *nearest other* cluster. $s_n \in [-1, 1]$: close to $+1$ means x_n sits comfortably inside its own cluster; close to 0 means it sits on a cluster boundary; negative means it is probably in the wrong cluster. The overall silhouette score is $\frac{1}{N}\sum_n s_n$. Higher is better. Unlike inertia, the silhouette score has a maximum at the "right" k.

(i) **[4 pts]** For each $k \in \{2, 3, \ldots, 10\}$, run k-means once (best of 10 random inits), compute the silhouette score (`sklearn.metrics.silhouette_score`), and plot silhouette vs k.

(ii) **[3 pts]** Which k maximises silhouette? Does it agree with the elbow from part (e)? If they disagree, which would you trust more here, and why?

(g) **Sensitivity to initialization. [6 pts]**

Lloyd's algorithm only finds a *local* minimum of J; the local minimum it lands in depends on the initial centroids.

(i) Run k-means with $k = 3$ for 50 different random seeds. Plot a histogram of the final inertias. How wide is the spread?

(ii) For two specific seeds (one giving the lowest inertia, one giving the highest), make side-by-side (`flipper_length_mm`, `bill_length_mm`) scatter plots coloured by cluster. Do the two clusterings disagree on a few points or radically?
Reflection: Real implementations always run k-means multiple times and keep the lowest-J run. Smarter initializers (`k-means++`) reduce this variability further—but the underlying point is that any clustering you obtain is one of many valid local minima.

(h) **Summary and recommendation. [5 pts]**

In 4–6 sentences, write the paragraph you would put in the lab's report:

- How many groups did you find, and how confident are you in that number (referencing both elbow and silhouette)?
- How well did the unsupervised clustering recover the recorded species labels?
- Name one limitation of this analysis (e.g. data is from a small geographic region, four morphological features only, missing values were dropped rather than imputed, ...).

Problem 4 (Dimensionality Reduction: PCA from Scratch vs. UMAP on Cancer Gene Expression)

A bioinformatics lab has just sent you a CSV. Each row is one tumor sample from a real patient; each column is the expression level of one human gene. There are 801 samples and 20,531 genes. Each tumor has been classified by pathologists into one of five cancer types: breast (BRCA), kidney renal clear cell (KIRC), colon adenocarcinoma (COAD), lung adenocarcinoma (LUAD), and prostate adenocarcinoma (PRAD). The lab's question: *can we visualise these tumors in 2D and "see" the cancer types separate, without using the labels?* If yes, that is a strong signal that gene expression alone carries enough information to type a tumor—useful for diagnostics, drug discovery, and basic research.

You will use **Principal Component Analysis (PCA)** and **Uniform Manifold Approximation and Projection (UMAP)** to answer this. PCA is a linear method: it finds the directions of maximum variance. UMAP is a nonlinear method built on differential geometry of fuzzy simplicial sets and stochastic gradient descent on a graph—which is why we will not implement it from scratch. Implementing PCA *is* short, and you will write it two ways and prove they agree.

Tools. PCA must be implemented from scratch in NumPy. For UMAP you may use the `umap-learn` library (`pip install umap-learn`, then `import umap`). You may use `sklearn.linear_model.Log` for the downstream classifier in part (i).

Dataset. Download the *Gene Expression Cancer RNA-Seq* dataset from the UCI Machine Learning Repository:

https://archive.ics.uci.edu/dataset/401/gene+expression+cancer+rna+seq

The dataset arrives as two files inside the downloaded archive:

- `data.csv`: 801 rows, 20,532 columns (one ID column plus 20,531 genes).
- `labels.csv`: 801 rows, 2 columns (ID and tumor type).

The 5 classes:

BRCA	Breast invasive carcinoma	LUAD	Lung adenocarcinoma
KIRC	Kidney renal clear cell	PRAD	Prostate adenocarcinoma
COAD	Colon adenocarcinoma		

(a) **Look at the data.**

(i) **[3 pts]** Load `data.csv` and `labels.csv`. Drop the ID column from `data.csv`. Confirm the shape is $801 \times 20{,}531$. Report the count of samples in each tumor class.

(ii) **[3 pts]** Briefly inspect the values: print `X.describe()` (or its summary on a few random gene columns). These are RNA-Seq expression values—typically already normalised to a $\log_2$-like scale, so additional log transformation is *not* required. Report the approximate range you see.

(iii) **[2 pts]** Standardize the gene columns to zero mean and unit variance. Why standardise here? (Hint: see Problem 4(b)(iii) of Homework 1; the same logic applies.)

(b) **PCA via SVD.**

Recall: For a centered data matrix $X_c \in \mathbb{R}^{N \times d}$, the singular value decomposition is

$$X_c = U \Sigma V^T,$$

where the columns of V are the principal axes. Projecting onto the top k components gives the low-dimensional representation $X_c V_{:,1:k}$.

(i) **Implement. [6 pts]** Write a function

```
def pca_svd(X, k):
    # returns components (k, d), projected (N, k), explained_variance (k,)
```

Inside the function: center X, call `np.linalg.svd(X_c, full_matrices=False)`, take the top k rows of V^T, and project X_c onto them. The variance explained by component i is $\sigma_i^2/(N-1)$, where σ_i is the i-th singular value.
Note on shapes: on this dataset $N = 801$ and $d = 20{,}531$; `full_matrices=False` is essential, otherwise NumPy will try to materialize a $20{,}531 \times 20{,}531$ matrix.

(ii) **[2 pts]** Run `pca_svd(X, k=2)`. Report the variances explained by the first two components.

(c) **PCA via covariance eigendecomposition.**

Recall: The principal axes are also the eigenvectors of the covariance matrix

$$C = \frac{1}{N-1} X_c^T X_c$$

sorted by descending eigenvalue.

(i) **Why this is impractical here. [3 pts]** The covariance matrix on this dataset is $20{,}531 \times 20{,}531$ floats ≈ 3.4 GB. Most laptops cannot allocate that. Use the *Gram trick*: when $N \ll d$, compute instead the small Gram matrix

$$G = \frac{1}{N-1} X_c X_c^T \quad (N \times N),$$

eigendecompose G, and recover the principal axes via $v_j = X_c^T u_j / \sqrt{(N-1)\lambda_j}$ for each top eigenpair (λ_j, u_j) of G. Eigenvalues of G are the same as the nonzero eigenvalues of C. Implement

```
def pca_eig(X, k):
    # returns components (k, d), projected (N, k), explained_variance (k,)
```

using `np.linalg.eigh` on the small $N \times N$ Gram matrix. Be careful: `eigh` returns eigenvalues in *ascending* order, so reverse them before taking the top k.

(ii) **Sanity check. [5 pts]** Run `pca_eig(X, k=2)` and compare with `pca_svd(X, k=2)`:

- Are the explained variances the same (up to floating-point error)?
- Are the principal axes the same up to a sign flip? (Eigenvectors are only defined up to sign—both v and $-v$ are valid axes.)

Report whichever check confirms agreement.
Why this check matters: the equivalence between SVD-of-data and eigendecomposition-of-covariance (or its Gram dual) is a theorem you proved in Problem 2(2) of this homework. When the two implementations disagree by more than floating-point noise, you have a bug.

(d) **Scree plot.**

(i) **[4 pts]** Run PCA with $k = 100$ (or the maximum allowed by $\min(N, d) - 1 = 800$, whichever is smaller). Plot the explained variance of each component, and on a separate axes plot the cumulative fraction of variance explained.

(ii) **[3 pts]** How many components are needed to explain 80% of the variance? How does this compare to the original 20,531 genes?
Why this is striking: the data has *intrinsically* much fewer than 20,531 dimensions of variation—most genes either don't vary across these samples or co-vary with each other. The scree plot makes this concrete.

(e) **2D PCA visualization. [5 pts]**

Project all 801 samples to 2D using your PCA implementation. Make a scatter plot, colouring each point by its tumor type (5 colours, with a legend). Comment in 2–3 sentences: do tumors of the same type cluster together? Which classes does PCA *succeed* at separating in 2D, and which does it confuse?
Hint: Different tumor types arise from different tissues, but RNA-Seq expression patterns can be more similar between, say, two adenocarcinomas (LUAD and COAD) than between either of them and BRCA. Linear projections may not be enough to disentangle them.

(f) **What does PC1 mean?**

A principal component is just a vector in gene space. Its *loading* on each gene tells you which genes drive that direction of variance.

(i) **[4 pts]** Take the first principal component (a vector of length 20,531). Report the indices and values of the 10 genes with the largest absolute loadings. These are the genes most responsible for the direction of greatest variance in the dataset.

(ii) **[3 pts]** Repeat for the second principal component. Are the top genes for PC1 and PC2 different?
Why this matters: this is what makes PCA *interpretable*. In a real bioinformatics workflow you would now look up these genes by ID and check whether they are known cancer markers—PC1 is whatever those genes encode together.

(g) **UMAP via library.**

Recall: UMAP is a nonlinear dimensionality-reduction method. Unlike PCA, it does not seek directions of maximum variance; instead it builds a fuzzy similarity graph on the data and then optimises a 2D embedding that preserves the graph's local structure. In practice this means UMAP is very good at separating classes that PCA fails on, at the cost of distorting global distances.

(i) **[5 pts]** Install `umap-learn` (`pip install umap-learn`). Fit a UMAP embedding to 2D using `umap.UMAP(n_components=2, random_state=42)`. Make a scatter plot of the 2D embedding coloured by tumor type.

(ii) **[3 pts]** Compare side-by-side with the PCA scatter from part (e). Which method gives more visually separable classes? Why might that be—what flexibility does UMAP have that PCA does not?

(h) **UMAP hyperparameter exploration. [5 pts]**

UMAP has two hyperparameters that strongly affect the embedding:

- `n_neighbors` (default 15): how many local neighbours each point's neighbourhood is built from. Smaller values emphasise very local structure; larger values capture more global structure.
- `min_dist` (default 0.1): minimum distance between points in the embedding. Smaller values make tighter clumps.

Make a 2×3 grid of UMAP scatter plots for the 6 combinations

$$(\texttt{n\_neighbors}, \texttt{min\_dist}) \in \{5, 15, 50\} \times \{0.0, 0.5\}.$$

In 2–3 sentences, describe how the embedding changes as these are swept. Which corner of the grid would you ship to the bioinformatics lab?

(i) **Downstream task: does the embedding actually help? [8 pts]**

A 2D scatter plot is suggestive but not proof. The honest test: *does a classifier trained on the low-dim embedding match the performance of one trained on the full gene-expression vector?*

(i) Split your 801 samples into 80% train / 20% test (stratified by class so every class is represented in test).

(ii) Train three logistic regression classifiers (`LogisticRegression(max_iter=2000)`):
- on the full 20,531-gene vector;
- on the 2D PCA embedding from part (e);
- on the 2D UMAP embedding from part (g).

(iii) Report test accuracy for each. Does the 2D embedding (PCA or UMAP) come close to the full-feature baseline? Which 2D method is closer? Reflect briefly: would $k = 10$ PCA components do even better?

(j) **Summary. [3 pts]**

Single table summarising: method, dimensions, downstream test accuracy. In 2–3 sentences, recommend one approach to the bioinformatics lab and explain when you would and would not use it.

Problem 5 (Decision Trees on Bank Marketing Data)

A Portuguese bank ran a series of telemarketing campaigns between 2008 and 2010, calling clients to pitch them a long-term term deposit. The campaigns were expensive: each phone call costs employee time and annoys clients who are not interested. The bank's analytics team wants to know, *before they place the call*, which clients are most likely to subscribe—so they can prioritise calls to those clients and stop wasting calls (and goodwill) on the rest.

The bank wants two things from you. First, a model that predicts subscription from the client's demographic and contact-history features—accuracy matters, but interpretability matters more, because the model has to be readable by people who are not machine-learning specialists. Second, an answer to: "*which features are doing the heavy lifting?*" This is exactly what a **decision tree** gives you—a model that is itself just a sequence of yes/no questions, and that exposes feature importance for free.

Tools. `pandas` for data wrangling; `sklearn.tree.DecisionTreeClassifier`, `sklearn.linear_model.Logist` and `sklearn.metrics` are all allowed. Implementing a tree from scratch is *not* required.

Dataset. Download the *Bank Marketing* dataset from the UCI Machine Learning Repository:

https://archive.ics.uci.edu/dataset/222/bank+marketing

The download contains several files; use `bank-full.csv` (45,211 clients, semicolon-separated). Columns:

Column	Type	Description
`age`	Numerical	Client's age
`job`	Categorical	Type of job (admin, blue-collar, technician, ...)
`marital`	Categorical	Marital status
`education`	Categorical	primary, secondary, tertiary, unknown
`default`	Categorical	Has credit in default?
`balance`	Numerical	Average yearly balance (EUR)
`housing`	Categorical	Has housing loan?
`loan`	Categorical	Has personal loan?
`contact`	Categorical	Contact communication type
`day`	Numerical	Last contact day of the month
`month`	Categorical	Last contact month of the year
`duration`	Numerical	**Last contact duration in seconds (*see warning below*)**
`campaign`	Numerical	# contacts during this campaign
`pdays`	Numerical	Days since client was last contacted (-1 = never)
`previous`	Numerical	# contacts before this campaign
`poutcome`	Categorical	Outcome of previous campaign
`y`	**Target**	yes / no – did the client subscribe to a term deposit?

Important warning about `duration`. `duration` is the length of the phone call *in which the offer was made*—which means it is only known *after* the call has happened. A model that uses `duration` as an input is using a feature that did not exist at the moment the prediction would be made in production. This is called **target leakage**, and including it would inflate your accuracy in a way that does not transfer to deployment. **Drop `duration` before training any model.**

(a) **EDA.**

(i) **[3 pts]** Load `bank-full.csv` (`sep=';'`). Print shape, dtypes, and the first 5 rows. How many features are numerical and how many categorical?

(ii) **[3 pts]** What fraction of clients have `y == 'yes'`? Is this a balanced classification problem? What accuracy would the trivial "always predict no" classifier achieve?

(iii) **[2 pts]** Drop `duration` from the dataset, citing the leakage warning above.

(iv) **[4 pts]** For two interesting features (`age` and `balance`), plot histograms split by the target y (overlaid with transparency). Briefly describe what you see. Repeat for one categorical feature (`poutcome`) using a bar chart of subscription rate per category.

(b) **Preprocessing.**

(i) **[3 pts]** Encode the target: `yes` $\to 1$, `no` $\to 0$. Split into 60% train / 20% validation / 20% test using `np.random.seed(42)`. Verify the class balance is preserved across splits.
Note: this dataset has no missing values in the `NaN` sense, but several categorical columns have `'unknown'` as a category—treat `'unknown'` as its own category rather than imputing.

(ii) **[4 pts]** One-hot encode all categorical features using `pd.get_dummies`, dropping one category per column to avoid multicollinearity. How many features do you end up with after encoding?
Note: Decision trees do not strictly require one-hot encoding (they can split on a categorical natively), but `sklearn`'s `DecisionTreeClassifier` does not handle string categoricals directly, so we one-hot encode anyway.

(c) **Fit a decision tree.**

(i) **[3 pts]** Fit `DecisionTreeClassifier(max_depth=3, random_state=42)` on the training data. Report training and validation accuracy.

(ii) **[5 pts]** Visualize the tree using `sklearn.tree.plot_tree` (with `feature_names` so the splits are human-readable). Read the top three splits aloud as if-then-else rules in English (e.g. "if `poutcome_success` ¿ 0.5 then ..."). Each split was chosen by maximising the Gini gain you analysed in Problem 3(2) of this homework.
Why interpretability matters: a depth-3 tree is something a non-ML person can verify by hand—"does that rule make sense?" Once the tree is depth 20, that ability is gone.

(d) **Overfitting and depth.**

(i) **[5 pts]** Sweep `max_depth` $\in \{1, 2, 3, 5, 7, 10, 15, 20, \text{None}\}$ (where `None` means "unlimited"—grow until every leaf is pure or below `min_samples_split`). For each depth, report training and validation accuracy.

(ii) **[4 pts]** Plot training and validation accuracy on the same axes as a function of depth. At what depth does validation accuracy peak? After the peak, does training accuracy keep improving while validation degrades? Describe what you see in 2–3 sentences in terms of bias and variance.

(e) **Cross-validation for hyperparameter tuning.**

A single train/val split can give a noisy estimate of the best depth. Use 5-fold cross-validation on the training set to make this more robust.

(i) **[6 pts]** For each `max_depth` from part (d), perform 5-fold cross-validation on the training set (`sklearn.model_selection.KFold` or `cross_val_score`). Report mean and standard deviation of validation accuracy across folds.

(ii) **[3 pts]** Pick the depth that maximises mean cross-validated accuracy. Refit at this depth on the full training set, and report **test** accuracy. Does it match what you would have picked from a single split in part (d)?

(f) **Other regularization knobs. [6 pts]**

`max_depth` is not the only way to prevent overfitting. Two others:

- `min_samples_split`: minimum samples a node must have to be split.
- `min_samples_leaf`: minimum samples in any leaf.

Sweep `min_samples_leaf` $\in \{1, 5, 20, 100, 500\}$ at the best depth from part (e). Plot validation accuracy vs. `min_samples_leaf`. Pick the best value and report combined-test accuracy at that combination.

(g) **Feature importance.**

(i) **[4 pts]** For your best tree from part (f), extract `tree.feature_importances_` and plot the top 15 features by importance.

(ii) **[3 pts]** Which features dominate? Do they match patterns you saw in the EDA in part (a)(iv)? In particular, did the tree pick up on `poutcome`, `contact`, `month`, `age`?

(h) **Compare to logistic regression. [5 pts]**

Train a logistic regression baseline on the same one-hot-encoded features. Standardize the numerical features first. Report test accuracy and compare to your best tree. Note: which model is faster to train, which is easier to explain to a non-specialist, and which would handle a new categorical level (e.g. a new `job` category) more gracefully?

(i) **Beyond accuracy. [5 pts]**

The bank's actual goal is not to maximise accuracy—it is to *spend its calls well*. Think about the two error types:

- **False positive:** predicted "will subscribe," actually said no. Cost: one wasted phone call.
- **False negative:** predicted "will not subscribe," actually would have said yes. Cost: one lost subscription.

For your best tree, compute the **precision** and **recall** on the test set. In 3–4 sentences, discuss: which error type does the bank care about more? Would you tune the prediction *threshold* (the cutoff above which the model predicts "yes") up or down to better serve them?

(j) **Reflection. [3 pts]**

In 2–3 sentences, name one thing you would try next to improve performance (random forests, gradient boosting, calibrated probabilities, ...) and one thing you would investigate before deploying this model in production.